

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 1

Выполнила:

Стринадкина Полина

Группа

J3213

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Подключить ранее разработанный интерфейс (ЛР1) к внешнему API, реализованному с помощью JSON Server. Обеспечить загрузку данных, навигацию между страницами и базовую авторизацию пользователей.

Ход работы

1. Был установлен Node.js и инициализирован npm-проект с помощью команды `npm init -y`. Сервер был настроен и запущен, что обеспечило доступ к API по адресу <http://127.0.0.1:4000>.

```
PS D:\ITMO-AIT-Frontend-2026\J3213\Стринадкина_Полина\labs\lab2\online-courses> npm start
Debugger attached.

> online-courses@1.0.0 start
> json-server db.json -m ./node_modules/json-server-auth -p 4000 --host 127.0.0.1

Debugger attached.

\{^_}/ hi!

Loading db.json
Loading ./node_modules/json-server-auth
Done

Resources
http://127.0.0.1:4000/users
http://127.0.0.1:4000/courses

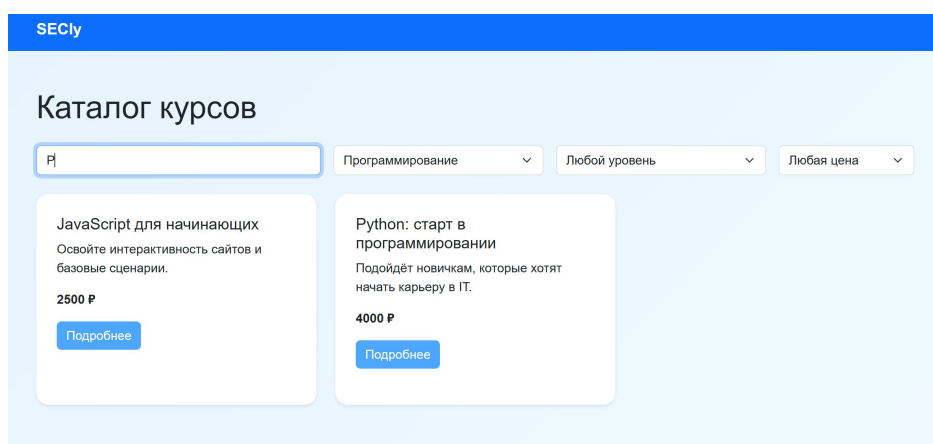
Home
http://127.0.0.1:4000

Type s + enter at any time to create a snapshot of the database
```

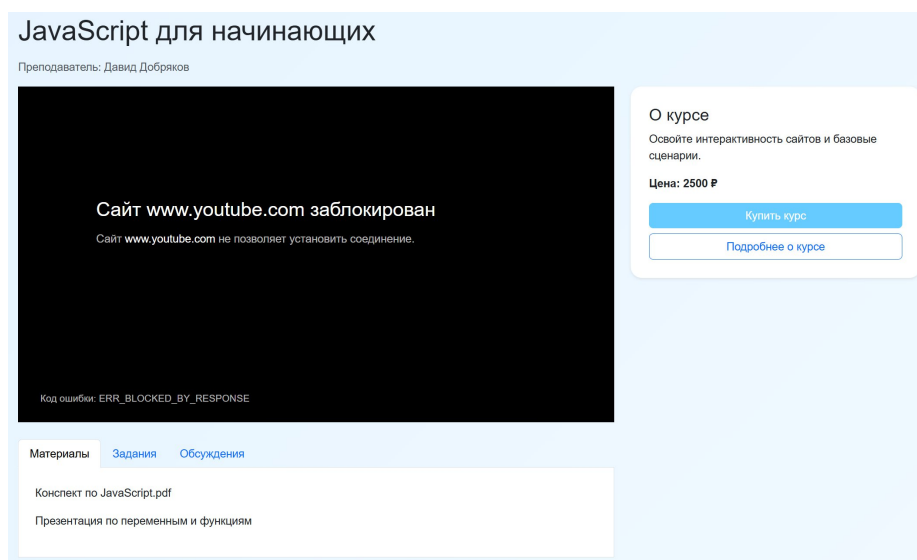
2. Далее был создан файл `db.json`, база данных. В нем были описаны пользователи и курсы. Для курсов были добавлены поля, отражающие их структуру (название, описание, преподаватель, цена, уровень, категория, материалы, задания и обсуждения). Для пользователей были реализованы поля `email`, `password`, `role` и дополнительные данные, что позволило имитировать систему авторизации.

```
{
  "users": [
    {
      "email": "strinadkinapolina@mail.com",
      "password": "$2a$10$1wT3gR3bIDgUk5jmVn75beMPNgfk4jv3iEJAXlapUW4g4GQHnvMiS",
      "name": "Полина",
      "role": "student",
      "myCourses": [],
      "certificates": [],
      "id": 1
    },
    {
      "email": "dobryakovdavid@mail.com",
      "password": "$2a$10$f4wmJtGcOHGEnCTPWQ/zr.RFh7ooBF39McUYLwayoDN7Mly6PZwNS",
      "name": "Давид",
      "role": "teacher",
      "myCourses": [],
      "certificates": [],
      "id": 2
    }
  ]
}
```

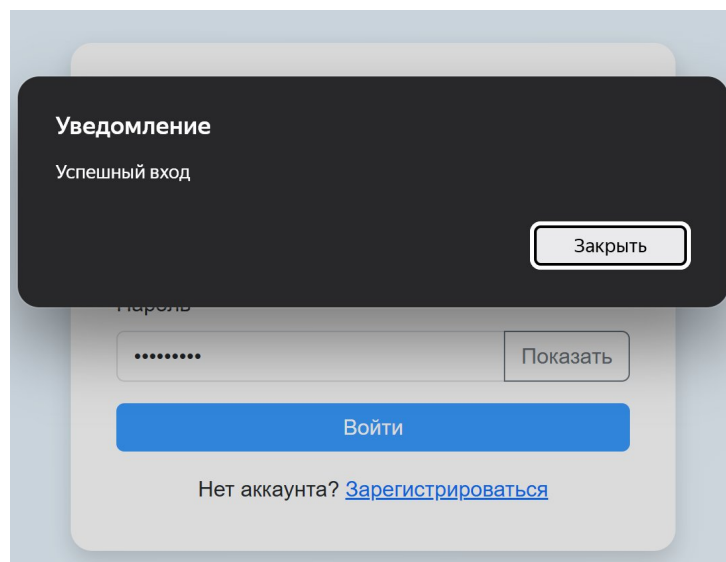
3. Подключено API к интерфейсу. Для выполнения HTTP-запросов была использована библиотека `axios`. С её помощью реализована загрузка списка курсов, получение данных конкретного курса по `id`, а также регистрация и вход пользователей.
4. Реализована фильтрация курсов по различным параметрам: названию, категории, уровню и цене. Фильтрация выполняется на стороне клиента после получения данных с API, что позволяет мгновенно обновлять отображаемый список.



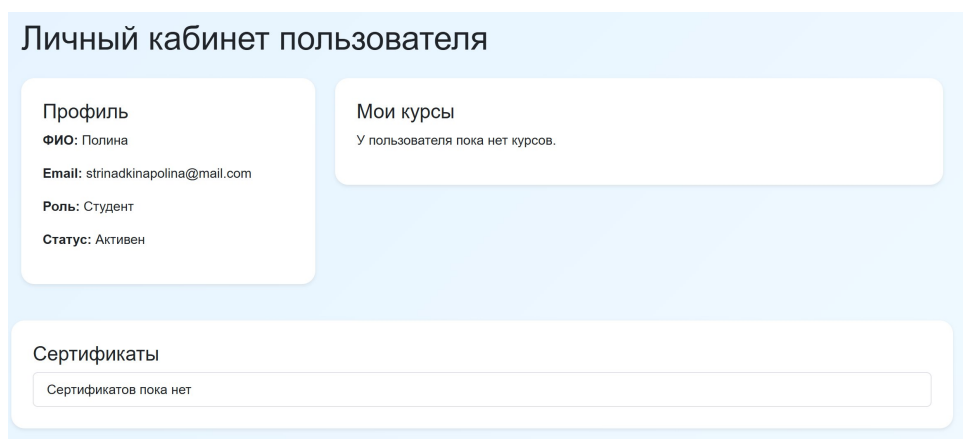
5. Для отображения информации о конкретном курсе была реализована отдельная страница, которая работает динамически. При переходе на неё передаётся `id` курса через URL, после чего выполняется запрос к серверу, и страница заполняется соответствующими данными. Это позволило использовать одну страницу для всех курсов. Также была реализована навигация между страницами приложения. При нажатии на кнопку «Подробнее» происходит переход на страницу курса с передачей идентификатора, что обеспечивает корректное отображение выбранного курса.



6. Отдельное внимание было уделено реализации авторизации. Пользователь может зарегистрироваться, после чего его данные сохраняются в базе. При входе происходит проверка введенных данных. После успешной авторизации информация о пользователе сохраняется в localStorage, что позволяет использовать её на других страницах приложения.



7. На основе данных из localStorage реализованы личные кабинеты пользователя и преподавателя. В них отображается информация о пользователе. Также реализован функционал выхода из системы.



Вывод

В ходе выполнения лабораторной работы были освоены принципы взаимодействия клиентской части приложения с API. Был реализован моковый сервер, обеспечивающий хранение и обработку данных. Получены навыки работы с HTTP-запросами, динамической отрисовкой интерфейса, фильтрацией данных и хранением информации на стороне клиента.